Let’s follow step by step the example on figure 9.9.

Listing 9.3 The search method

```
function search(node,
target)                                                    #1
  if node == null
then                                                           #2
    return null
  elsif node.point == target
then                                                 #3
    return node
  elsif compare(target, node) < 0
then                                             #4
    return search(node.left        , target)

else
     #5
    return search(node.right, target)
```

#1 Search returns the tree node that contains a target point, if the point is stored in the tree – returns `null` otherwise. We explicitly pass the root of the (sub)tree we want to search, so we can reuse this function for sub-trees.

#2 If node is already `null`, we are traversing an empty tree, and by definition it does not contain any point.

#3 If the target point matches this node’s point, then we have found what we were looking for.

#4/#5 Otherwise, we need to compare the right coordinates of target and node’s point: we use the helper method previously defined, and check if it is lower than `0`, which would mean we have to take a left turn during tree traversal, and therefore we run the method recursively on the left or right branch.